2017 Workshop on Research, Education and Development of
Unmanned Aerial Systems (RED-UAS)
Linköping, Sweden, October 3-5, 2017

# Hand Gesture Recognition with Convolutional Neural Networks for the Multimodal UAV Control

Yuntao Ma[1], Yuxuan Liu[1], Ruiyang Jin[1], Xingyang Yuan[1], Raza Sekha[1], Samuel Wilson[1], Ravi Vaidyanathan[1]

*Abstract*— **We introduce a robust wearable sensor suite fusing arm motion and hand gesture recognition for operator control of UAVs. The sensor suite fuses mechanomyography (MMG) and an inertial measurement unit (IMU) to capture multimodal (arm movement and hand gesture) command signals simultaneously. The IMU produces world referenced orientation and acceleration data while concomitant MMG tracks muscle activation through surface vibration. The use of surface muscle vibration for gesture recognition removes the need for electrical contact with the skin, which has impeded other forms of muscle measurement for gesture recognition in the field. This investigation presents hardware design, inertial recognition of arm movement, and the detailed structure of a convolutional neural network (CNN) system used for real-time hand gesture recognition based on MMG signals. The system achieved 94% accuracy for five gestures with simple calibration for each user, thereby providing an intuitive gesture based UAV control system. To our knowledge this is the first wearable system enabling multimodal control of UAVs through intuitive gestures that does not require electrical skin contact. Future work involves testing the system with larger UAV swarms.**

## I. Introduction

Despite significant advances in unmanned air vehicle (UAV) autonomy, human operation is still necessary for utility in many unstructured environments, particularly in situations where a drone must interface closely with people. Control of UAVs tends to fall into two categories, low-level and high-level control [1]. Low-level control normally consists of a closed loop controller onboard which performs tasks such as UAV stabilization. High-level direction involves a pilot controlling the movement of the UAV, actuated by either directly influencing the operation of the UAV, or by applying biases to the low-level control system. Common control methods for UAVs include radio control from a third person view, piloting through a flight console, and first-person view (FPV) piloting [2]. Many works have also introduced inertial-based control of UAVs, where a measurement unit is moved in a manner reflecting desired flight motion. These methods all require control devices occupying both hands. Push buttons or other forms of input are also typically required for additional UAV actions.

Gesture-based control has been explored extensively as an alternative to hand-held joysticks. Commercial gesture controllers for UAVs are mostly vision based systems mounted on the UAV (e.g. the DJI Spark [3]), combining manual control and semi-automation for stabilization and hovering. Vision-based gesture recognition for UAV control has also be implemented using Microsoft Kinect [4]. Benefits of vision systems include easily accessible commercial hardware, and rapid acquisition of world referenced movements; however, these approaches also introduce a number of limitations. For UAV-mounted-camera recognition systems, the user who controls the UAV must have the camera on UAV facing them, which limits the range of operation

Wearable systems for gesture recognition have been proposed to surmount these issues. These systems commonly use electromyography (EMG), which monitors the electrical signals produced during muscle activation. Many systems have also attempted to fuse inertial measurement with gesture recognition, where UAV motion can be controlled with arm movement and specific actions are invoked through gestures. IMU and EMG based systems have been used successfully to control mobile robots [5]. Surface EMG signals are known to suffer from transient calibration variables, caused by factors such as changing sweat level[6]. This imposes limitations on long term applications of EMG based gesture recognition systems.

Mechanomyography (MMG) offers an alternative method for monitoring muscle activity. MMG involves the measurement of low frequency (2-200 Hz) vibrations emitted by skeletal muscle which are produced during contraction. Its signal is comprised of the gross mechanical activity of muscle, and the lateral oscillation of muscle fibres [7]. Although MMG measurement is less mature and suffers from a lack of established sensors and processing practice, it nevertheless offers potential benefits when compared to EMG. These include a higher signal-to-noise ratio (SNR), multiple uses for a single sensor , and elimination of the need for shaving and conductive gel [8], [9]. MMG also does not require a reference sensor, the value read from the sensor is an absolute measurement of the pressure waves, eliminating the reference value as a potential source of errors. It has been demonstrated to be a more robust alternative to EMG in medical applications such as gait analysis [10] and prosthetic hand control [11].

In this work, we present a multimodal arm motion and gesture recognition control system for high-level UAV teleoperation. The system overcomes issues associated with using EMG outside controlled environments through the implementation of MMG based gesture control fused with inertial sensors for arm movement. Our system marries high and low-level control, thereby providing more intuitive teleoperation.

*This research was funded by the US Office of Naval Research Global (ONR-G) grant N62909-14-1-N221 and the UK-India Educational Research Initiative (UK-IERI) under award IND/CONT/E/14-15/366.

[1]Department of Mechanical Engineering Imperial College London, London, UK r.vaidyanathan@imperial.ac.uk

978-1-5386-0938-5/17/$31.00 ©2017 IEEE


Authorized licensed use limited to: ISAE. Downloaded on November 05,2026 at 10:38:34 UTC from IEEE Xplore. Restrictions apply.

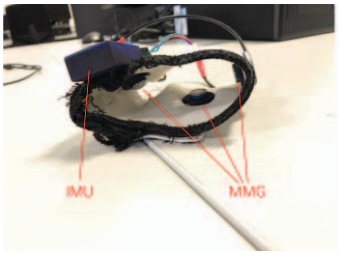



Fig. 1. Arm band with 1 IMU and 3 MMGs

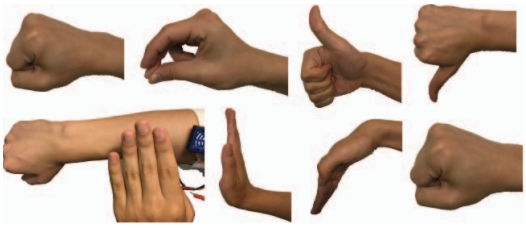


Fig. 2. Gestures used. Row 1 Left to Right, 1. Clenching Fist, 2. Snapping, 3. Thumb Up, 4. Thumb Down. Row 2 Left to Right, 5. Double Tap on Forearm, 6. Palm Up, 7. Palm Down, 8. Double Clenching Fist

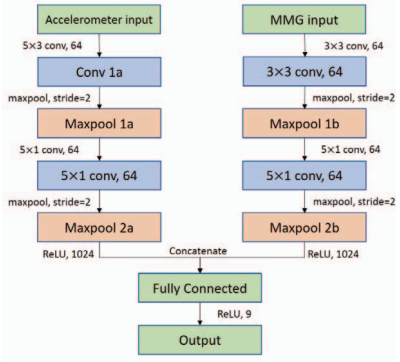



Fig. 3. The neural network structure used in this experiment

### A. Summary of Work

Gesture classification was performed on eight distinct gestures. The hardware used was an armband consisting of one IMU and three custom made MMG sensors, previously introduced for medical monitoring [12]. Data from the MMGs were used for gesture recognition and the IMU provided arm movement detection. The armband was placed on forearm over the brachioradialis, flexor carpi radialis and extensor carpi ulnaris muscles. During the experiment, two Crazyflie 2.0 quad-copters [13] and three webcams were used.

A convolutional neural network (CNN) was chosen to classify the gesture signals for two reasons: Firstly, it was observed that the temporal position of the features within the identified gesture window was not constant. The CNN is able to overcome this by assuming that proximate features are connected regardless of their overall position in the dataset. The CNN achieves this by connecting input neurons in the same receptive field to produce neural activation for the next layer. In this context, two convolutional layers were applied to the accelerometer and MMG data stream, so that local features in the MMG signals could be extracted. After this, each maxpool layer would take the maximum feature activation in adjacent time step, hence reduce the effect of the temporal offsets. Secondly, figure 4 shows the MMG signals of the same gesture collected from different participants. It was noted that each subject produced unique signal patterns for each of the test gestures , therefore it was necessary for individual participants to calibrate the network prior to use. A large number of parameters would need to be tuned if the full network was to be trained. However, as the convolutional layers could act as feature extractors, the calibration process could directly learn the mapping from the features to the classification, hence reducing the data size requirement. It was shown that 5 out of 8 gestures were classified with an average accuracy of 94.38% accuracy and a minimum accuracy of 85.4%.

This paper is organized as follows. Section II contains the Methodology, Section III contains the Results, Section IV and V contains our Discussion and Conclusions.

## II. Methodology

### A. Hardware Setup

The assembled sensor suite can be seen in Figure 1. A sample rate of 500Hz was chosen in order to avoid aliasing in the (2-200Hz) MMG signals. The IMU used in the experiment contained an accelerometer, a gyroscope and a magnetometer. By using a gradient descent algorithm to calculate the world referenced orientation [15], it is able to calculate roll, pitch and yaw angles to allow the interpretation of directional commands. The resolutions of the sensors are $\pm$2G, $\pm$500 degrees per second and $\pm$2 gauss, corresponding to the range of all 3 sub-components. The sensor data were streamed via Bluetooth to a computer, where processing and control signal generation occurred. Figure 2 above shows eight gestures used in this experiment.

### B. Convolutional Neural Network (CNN) Structure

Signal streams from the three MMGs and the 3-axis accelerometer on the IMU were fused to generate training datasets. These signals were compiled into a rolling buffer of length 300, allowing the previous 0.6s of data to be retained. This data buffer was then linearly standardized to values between +0.5 and -0.5 before being passed into the neural network. The network output is a one-hot vector with 9 dimensions. The first 8 dimensions correspond to the 8 gestures to be classified, and the last dimension corresponds to signals captured during directional arm movements including up, down, left and right (referred to as noise below, since the signals are not generated intentionally).

This CNN structure shown in Figure 3 is a minimized VGG net [16], characterized by the repeated application of convolutional and pooling layers. The convolutional layers were connected by the maxpool layers (1a, 1b, 2a, and 2b). Two convolutional layers were used to increase the field of reception of each neural node at the fully connected layers



Authorized licensed use limited to: ISAE. Downloaded on November 05,2026 at 10:38:34 UTC from IEEE Xplore. Restrictions apply.

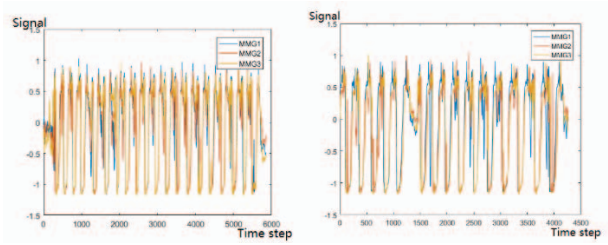



Fig. 4. Samples of MMG signal of fist clenching from two different participants, each of the peaks corresponds to clenching fist once

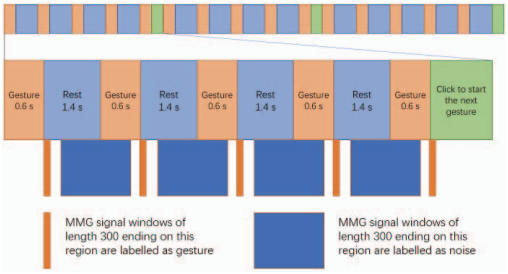



Fig. 5. Detailed calibration procedure

without significant increase in the number of parameters, and the maxpool layer reduces both the effect of signal shifting on the network prediction and the subsequent data size. ReLU is a computationally efficient non-linearity unit [17], and was used instead of sigmoid function or tanh to increase training and computation speed. The accelerometer and MMG signals are fed separately into two sub-networks, as they may have different optimal feature lengths and hence convolution size. The design parameters varied in this architecture included the number of convolutional and pooling layers, the width of each convolutional layer, the type of pooling layer and the type of non-linear unit. The setup that produced the highest accuracy and satisfied the processing speed requirement of under 0.002s per forward pass is shown in Figure 3.

The CNN was initially trained with a large dataset containing gesture signals previously recorded from 27 participants. The purpose of this step was to train the convolution layers as feature extractors, so that they were able to recognize the local patterns in the gesture signals. Initial weights and biases were set to random values distributed according to standard Gaussian distribution. Softmax [18] classifier is applied in the last fully-connected layer to compute the loss function. The loss function is minimized with respect to updated parameters (weights and biases) by using stochastic gradient descent optimization. During the calibration process, each user was asked to perform each of the gesture 8 times in response to a visual stimuli produced by the program. These gesture signals were then extracted and used to further train the neural network.

Dropout [19], a regularization method to prevent overfitting, was applied to the fully connected layer with a probability of 0.5 (a hyperparameter) in each forward pass. This produced the same outputs at the test time as the expected outputs at training time. Another method to prevent overfitting is to apply data augmentation on the original dataset to enlarge the size of the dataset while preserving the labels prior to training the machine learning classifier. As there are only limited training examples, the classifier will perform better with lower generalization error. Here, translation was chosen to manipulate the dataset, each gesture input was shifted 5 times by 1 timestep . This allowed the training set to be expanded, leading to a potential improvement in the performance of CNN.

### C. Calibration Process

To calibrate the classifier for each individual, the user was given a visual cue via a C# windows application form through a progress bar, signifying that they should make the indicated gesture. A progress bar is used to prepare the user for performing the signal, so that the temporal offset of the MMG signals in the training set can be confined. The region of the MMG signals covering from 0.3s before the visual cue to 0.3s after is recorded, labelled with the gesture name and added to the training set. It is noted that this may not correspond exactly to the time periods that were manually identified during the offline training process, which is a source of the temporal offset . However, as each gesture was performed, the gesture samples for calibration would be located in similar location in the running window of 300 timesteps of data, so to ensure that the calibration did not lead to overfitting, each instance of a gesture was used to generate five data sets using the same data augmentation method as the offline training dataset. In this calibration phase, the learning rate was reduced from the offline training value (0.0005) to 0.0001, and the weights in the convolutional layers were fixed, meaning only the weights in the fully connected layers were trained. Both of these steps aimed to prevent the network from being overfitted on the small user-specific dataset. Noise data was also trained alongside the newly collected gesture signals at half the learning rate.

### D. Data Collection

Data were taken for the offline training of the neural network from 27 randomly chosen students from Department of Mechanical Engineering, Imperial College London. The data collection procedure was as follows: an armband with one IMU and 3 MMGs was worn by each participant on the right forearm, with IMU uppermost when the palm faces downwards. For each participant, 8 gestures in total were performed 15 times each, and 30 seconds of free movement was collected as training data for the noise class.

During data collection each participant was seated, holding the whole arm horizontal and straight to perform each gesture. All MMG data collected were saved in .csv files . After the raw data was processed, gestures were identified manually, and each period of 300 timesteps was labelled as the gesture input. Manually identifying each gesture was carried out by finding the large amplitude signals in the region where a gesture was expected, and marking the 0.6s



Authorized licensed use limited to: ISAE. Downloaded on November 05,2026 at 10:38:34 UTC from IEEE Xplore. Restrictions apply.

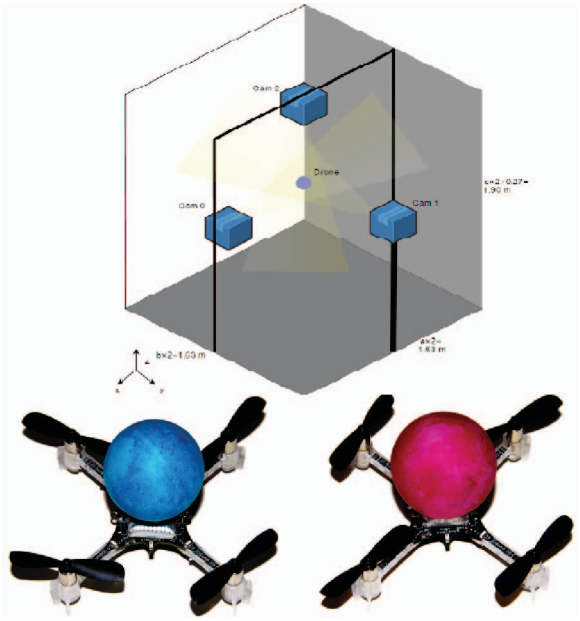



Fig. 6. Test Arena Setup (top) and Test UAVs, Crazyflie 2.0 (bottom)

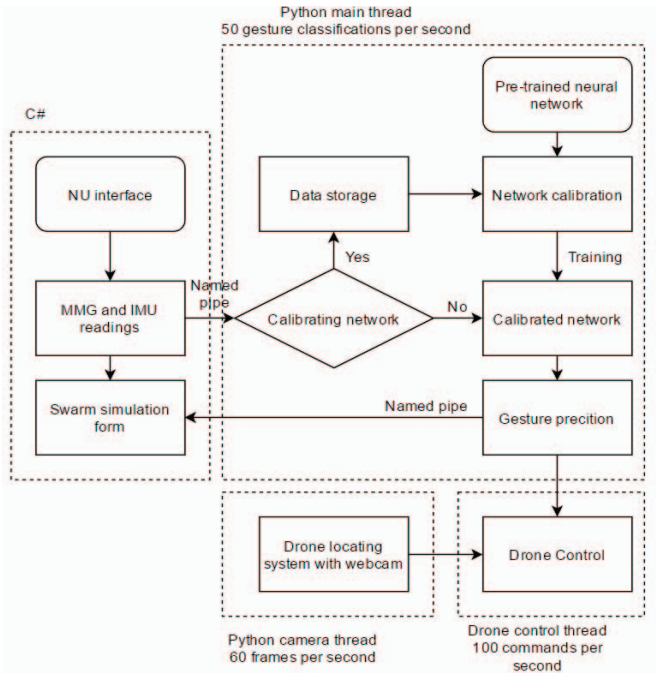



Fig. 7. Data flow diagram between C# application form and the python program, and among python threads

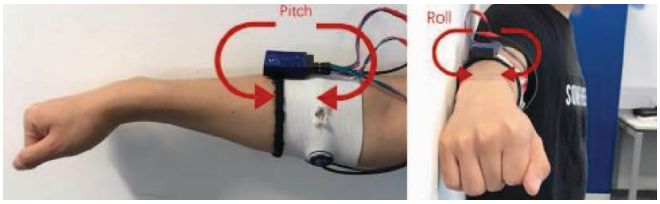



Fig. 8. Wrist band position and control method for translational movement

prior to the final trough in that area. Noise consists of non-gesture muscle movement; therefore, entire 30 seconds of noise data is fed to the neural network as training data. While controlling the UAV with the arm band, it was assumed that gesture performed occupies approximately 5% time of usage, hence 95% of time is noise, therefore, for isometric training, this percentage was used to distribute the training data (5% Gesture and 95% Noise).

Data from three other participants were used to test the accuracy of the calibrated neural network. Gesture signals from these participants were not used during the initial training. For each, 8 sample gestures were collected through the GUI. 5 of these gestures were used to generate the training data and the other 3 were used to generate the test data.

### *E. UAV (Quadcopter) Control System*

The quadcopters used in this research were Crazyflie 2.0s, and communication between the control system and the quadcopter was through Crazyradio [20]. The Crazyflie python library [21] was used to send high level commands including roll, pitch, yaw angles to the quadcopters.

The quadcopters were located optically with three 3 cameras located as shown in Figure 6. A uniquely coloured ball was attached to each quadcopter and declared as a drone class object. Once each drone is identified by all three cameras, the location of the drone within the arena is calculated. This tracking system enabled the controller to perform basic stabilization and hovering for the quadcopter, since they did not have this capability.

The target location is generated by the system based on the inertial sensors in the armband. The IMU controls the directional commands, and the gesture interpretation invokes the special functions of the system, such as switching the target for the controller from one quadcopter to another. Initially, target locations of both drones are locked. Clenching fist enables the target locations to move in the horizontal plane according to the directional commands set by the IMU. Snapping switches from controlling both quadcopters to controlling a single quadcopter. Opening palm changes the height of the quadcopters. A PID controller is then applied to move the drone to their target locations.

Data transmission between the C# form application and the CNN constructed on python is carried out with named pipes. A pipe server is created in the C# form and is fed with a byte array of the MMG signals as well as the roll, pitch, and yaw angle of the IMU. The python program reads the byte array information posted in the pipe by the C# form, passes it through the neural network and obtains the gesture classification. This is summarised in Figure 7.

A simplified CNN was trained and used to test the real-time control of two quadcopters with the armband due to the baud rate of the named pipe between C# GUI and the neural network running in python. The simplified CNN takes input length of 50 timesteps (instead of 300, with sampling frequency reduced by the factor of 6) with convolutional layers of depth of 32 (instead of 64).



Authorized licensed use limited to: ISAE. Downloaded on November 05,2026 at 10:38:34 UTC from IEEE Xplore. Restrictions apply.

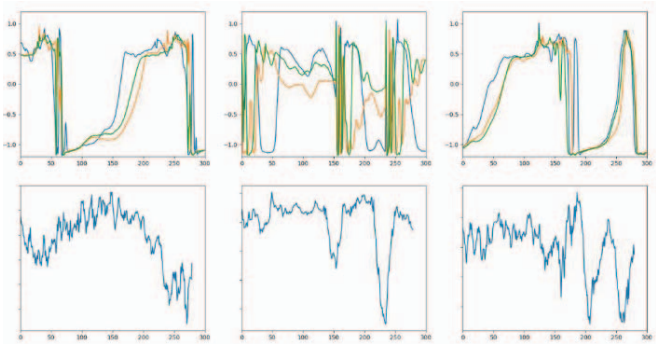


Fig. 9. Three input signals (top) (snapping, double tap, and double clenching) and their respective activation after a part of the signal is covered. For instance, in the middle image, the probability of the network recognizing the double tapping gesture dropped significantly after covering up either of the tapping signal. Conversely, occluding other regions have much smaller impacts.

## III. Results

Table I shows the confusion matrix of the intended gestures against the output classifications generated by the neural network. The accuracy of the calibrated neural network was tested with the data from the three test participants. The augmented test dataset contains 60 test samples of each gesture for each participant and another 1000 samples of noise is recorded. The diagonal terms indicate the accuracy that the gestures are correctly recognized. It is clear from the confusion matrix that network recognizes snapping accurately (100%), but gestures including thumb up and palm up are poorly recognized (40.6% and 49.5% correct). The network recognizes 98.8% of noise correctly, suggesting a good noise rejection rate.

Parts of the input can be systematically occluded to determine the part of the input that is most crucial in classification [22]. The input with a small patch set to 0 can be passed to the CNN, and the result is compared between the original input and the patched input. If the classification result has changed, it is implied that the covered region was responsible for the classification. This patch is iterated across all time steps in the input signal to compare the importance of the input regions. The results in Figure 9 show that the regions critical for gesture classification correspond intuitively, that the most indicative parts of the gesture usually correspond to the more vigorous muscle vibrations.

The forward time for the neural network on Intel Xeon E3-1230v5 CPU [23] was tested to be ≈0.0014s averaged over 500 trials, much faster than the data reception rate of once every 0.02 seconds, hence it is feasible to be used in real-time.

### A. Application to UAV Control

We were able to successfully control translational movement of UAVs, and have successfully sent preset command by performing gestures. This research has shown feasibility of multimodal gesture control system for multiple UAVs.

In the experiment, five gestures with at least 85% recognition accuracy were defined as feasible gestures, as shown in Table II. Due to limitation in UAV used in this experiment For demonstration purpose, three pre-defined commands were applied. These included switching control between two UAVs, locking/unlocking control of both UAVs and height adjustment, together with translational movement control with IMU. Other commands could easily be assigned for greater control.



Fig. 10. Photo taken during testing. Left, controlling translational movement with IMU. Right, performing gesture command.

The UAVs performed as expected, successfully received correct command sent by the user, and smooth translational movement control was achieved by using the IMU.

## IV. Discussion

There are many existing gesture recognition approaches using computer vision algorithms to detect the arm movement and classify the gesture classes. The methods were developed to construct a gesture-based human-robot interaction (HRI) for mobile robots [24]. These approaches often use spatial variation as data augmentation on the image-type training dataset to train the machine learning classifier. Some authors employed 3D CNN-based classifier that consists of a high-resolution network (HRN) and a low-resolution network (LRN). Their method achieved a gesture recognition rate of 77.5% on the VIVA challenge dataset [25]. Some other authors used a max-pooling CNN for vision-based gesture recognition were able to classify 6 gesture classes with 96% recognition accuracy [24]. We collected gesture input signal by using MMG sensors and IMU sensor to detect muscular activities and applied time-based variation as data augmentation on the signal-type training dataset

The experimental results show that the classifier model has potential to recognize hand gestures, which can be converted to commands and sent to the UAVs, enabling the user to direct complex flight tasks. A single gesture can be assigned a specific command for the UAV, significantly reducing complexity of UAV operation. For example, a cyclist followed by his own UAV for recording video can send command to it by performing single gesture without sacrificing safety. Gesture operation could also improve the immersion of the experience when flying a FPV drone with VR helmet.

## V. CONCLUSION

In this work, we presented a multimodal approach for controlling multiple quadcopters simultaneously. This approach uses gesture recognition from the MMG signals to adjust the quadcopters' response to the directional commands from the IMUs. With this approach, we successfully controlled two quadcopters, but the approach is not limited to the control of



Authorized licensed use limited to: ISAE. Downloaded on November 05,2026 at 10:38:34 UTC from IEEE Xplore. Restrictions apply.

TABLE I

CONFUSION MATRIX - GESTURES: 1. CLENCHING FIST, 2. SNAPPING, 3. THUMB UP, 4. THUMB DOWN, 5. DOUBLE TAPPING FOREARM, 6. PALM UP, 7. PALM DOWN, 8. CLENCHING FIST TWICE (LIKE DOUBLE CLICK), 9. NOISE

| | | Gesture Recognized | | | | | | | | | Total Gesture inputs |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | |
| Actual Gesture | 1 | 85.4% | 4.7% | 0.0% | 9.9% | 0.0% | 0.0% | 0.0% | 0.0% | 0.0% | 60 |
| | 2 | 0.0% | 100% | 0.0% | 0.0% | 0.0% | 0.0% | 0.0% | 0.0% | 0.0% | 60 |
| | 3 | 1.6% | 0.0% | 40.6% | 56.3% | 0.0% | 0.0% | 0.0% | 0.0% | 1.6% | 60 |
| | 4 | 9.9% | 0.0% | 0.0% | 90.1% | 0.0% | 0.0% | 0.0% | 0.0% | 0.0% | 60 |
| | 5 | 0.0% | 0.0% | 0.0% | 0.0% | 96.4% | 0.0% | 0.0% | 0.0% | 3.6% | 60 |
| | 6 | 0.0% | 0.0% | 0.0% | 0.0% | 0.0% | 49.5% | 0.0% | 0.0% | 50.5% | 60 |
| | 7 | 6.8% | 1.0% | 0.0% | 12.0% | 0.0% | 0.5% | 72.9% | 0.0% | 6.8% | 60 |
| | 8 | 0.0% | 0.0% | 0.0% | 0.0% | 0.0% | 0.0% | 0.0% | 100.0% | 0.0% | 60 |
| | 9 | 0.0% | 0.1% | 0.4% | 0.4% | 0.0% | 0.0% | 0.1% | 0.1% | 98.8% | 60 |
| | | | | | | | Total Accuracy | | | | 81.5% |

TABLE II

FEASIBLE GESTURES WITH PROPOSED COMMANDS

| Gestures No. | 5 | 4 | 2 | 1 | 8 |
|---|---|---|---|---|---|
| Gesture Movement | Double Tapping forearm | Thumb down | Snapping | Clenching Fist | Clenching Fist Twice |
| Recognition accuracy | 96.4% | 90.1% | 100% | 85.4% | 100% |
| Proposed Flight Task | Power on/off | Lock on/off all | Lock on quadcopter A/B | Rise the corresponding quadcopter | Lower the corresponding quadcopter |

UAVs can be extended beyond UAVs, to the control of other systems that requires intuitive input methods for high level commands. The gesture recognition is based on a CNN; it is able to classify four intended gestures, as well as noise with the accuracy of above 90%. The error rates of classifying random arm muscle movements to any gestures sum to 1.2%. Future works may include refining the neural network structure to simplify the calibration process and using other supervised learning algorithms to further increase the gesture classification accuracy.

## REFERENCES


[1] J. How and E. King, "Flight Demonstrations of Cooperative Control for UAV Teams," Architecture, vol. 1, Sep, pp. 505-513, 2004.

[2] J. D. Stevenson, S. O?Young, and L. Rolland, "Assessment of Alternative Manual Control Methods for Small Unmanned Aerial Vehicles," Procedia Manuf., vol. 3, no. Ahfe, pp. 952-959, 2015.

[3] DJI, "Spark - Seize the Moment." [Online]. Available: https://www.dji.com/spark. [Accessed: 22-Jul-2017].

[4] J. Bolin, J. Hoffman, C. Crawford, S. Beckmann, and W. Macke, "Gesture-Based Control of Autonomous UAVs ( Extended Abstract )," in Proc. of the 16th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2017), 2017, pp. 1484-1486.

[5] M. T. Wolf, C. Assad, M. T. Vernacchia, J. Fromm, and H. L. Jethani, "Gesture-based robot control with variable autonomy from the JPL BioSleeve," in 2013 IEEE International Conference on Robotics and Automation, 2013, pp. 1160-1165.

[6] B. S. Day, "Important Factors in Surface EMG Measurement," Measurement, pp. 1-17, 2002.

[7] M. J. Stokes and P. A. Dalton, "Acoustic myography for investigating human skeletal muscle fatigue," J. Appl. Physiol., vol. 71, no. 4, 1991.

[8] D. T. Barry, S. R. Geiringer, and R. D. Ball, "Acoustic myography: A noninvasive monitor of motor unit fatigue," Muscle Nerve, vol. 8, no. 3, pp. 189-194, 1985.

[9] M. J. Stokes, "Acoustic myography: Applications and considerations in measuring muscle performance," Isokinet. Exerc. Sci., vol. 3, no. 1, pp. 4-15, 1993.

[10] R. B. Woodward, S. J. Shefelbine, and R. Vaidyanathan, "Pervasive Monitoring of Motion and Muscle Activation: Inertial and Mechanomyography Fusion," in IEEE/ASME Transactions on Mechatronics, 2017, vol. XX, no. X, p. [In Press].

[11] S. Wilson and R. Vaidyanathan, "Upper-Limb Prosthetic Control Using Wearable Multichannel Mechanomyography," in IEEE International Conference on Rehabilitation Robotics (ICORR), 2017, p. 6.

[12] R. Woodward, S. Shefelbine, and R. Vaidyanathan, "Pervasive Motion Tracking and Muscle Activity Monitor," 2014 IEEE 27th Int. Symp. Comput. Med. Syst., vol. XX, no. X, pp. 421-426, May 2014.

[13] bitcraze, "Crazyflie 2.0 — Bitcraze." [Online]. Available: https://www.bitcraze.io/crazyflie-2/. [Accessed: 22-Jul-2017].

[14] J. Turian, L. Ratinov, Y. Bengio, and J. Turian, "Word Representations: A Simple and General Method for Semi-supervised Learning," Proc. 48th Annu. Meet. Assoc. Comput. Linguist., no. July, pp. 384-394, 2010.

[15] S. O. H. Madgwick and A. J. L. Harrison and R. Vaidyanathan, "Estimation of IMU and MARG orientation using a gradient descent algorithm", IEEE International Conference on Rehabilitation Robotics, pp. 1-7, 2011.

[16] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," arXiv Prepr., pp. 1-10, 2014.

[17] V. Nair and G. E. Hinton, "Rectified Linear Units Improve Restricted Boltzmann Machines," Proc. 27th Int. Conf. Mach. Learn., no. 3, pp. 807-814, 2010.

[18] X. Chen and Z. J. Wang, "Pattern recognition of number gestures based on a wireless surface EMG system," Biomed. Signal Process. Control, vol. 8, no. 2, pp. 184-192, 2013.

[19] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," J. Mach. Learn. Res., vol. 15, pp. 1929-1958, 2014.

[20] bitcraze, "Crazyradio PA — Bitcraze." [Online]. Available: https://www.bitcraze.io/crazyradio-pa/. [Accessed: 22-Jul-2017].

[21] bitcraze, "Overview — Bitcraze." [Online]. Available: https://www.bitcraze.io/development-overview/. [Accessed: 22-Jul-2017].

[22] M. D. Zeiler and R. Fergus, "LNCS 8689 - Visualizing and Understanding Convolutional Networks," 2014.

[23] Intel,"Intel® Xeon® Processor E3-1230 v5 (8M Cache, 3.40 GHz) Product Specifications." [Online]. Available: http://ark.intel.com/products/88182/Intel-Xeon-Processor-E3-1230-v5-8M–Cache-3_40-GHz. [Accessed: 23-Jul-2017].

[24] J. Nagi et al., "Max-pooling convolutional neural networks for vision-based hand gesture recognition," Signal Image Process. Appl. (ICSIPA), 2011 IEEE Int. Conf., pp. 342-347, 2011.

[25] P. Molchanov, S. Gupta, K. Kihwan, and J. Kautz, "Hand gesture recognition with 3D convolutional neural networks," Comput. Vis. Pattern Recognit. Work. (CVPRW), 2015 IEEE Conf., pp. 1-7, 2015.




Authorized licensed use limited to: ISAE. Downloaded on November 05,2026 at 10:38:34 UTC from IEEE Xplore. Restrictions apply.